



Tema central del Módulo 1

Estructuras lineales estáticas orientadas a C++

Aplicación al incidente crítico PAD-Bienestar

Este material explica cómo organizar, almacenar y manipular datos dentro de un programa en C++, usando arreglos, struct, arreglos de estructuras, listas estáticas, pilas, colas y archivos para persistencia.

Idea clave: No basta con guardar datos; hay que organizarlos de forma adecuada para buscarlos, modificarlos, recorrerlos, eliminarlos y conservarlos con eficiencia.

Sustento teórico

Deitel & Deitel, C++ Cómo programar, 9.^a edición

Caso de aplicación

Gestión de sesiones, turnos y bitácoras del PAD-Bienestar

1. Enfoque general: datos organizados en C++

En C++, una solución informática no se construye únicamente con variables aisladas. Cuando un sistema necesita manejar varios datos relacionados, como usuarios, equipos, sesiones activas, turnos o bitácoras, se requiere elegir una estructura de datos adecuada. Esta elección impacta en la claridad del código, la facilidad de mantenimiento y la eficiencia de operaciones como búsqueda, inserción, eliminación y recorrido.

Desde el enfoque de Deitel & Deitel, la programación en C++ promueve organizar los datos junto con las operaciones que se realizan sobre ellos, usando recursos como arreglos, estructuras, funciones y flujos de archivo. Para este módulo, el objetivo es construir una primera solución con estructuras lineales estáticas, entendiendo tanto sus ventajas como sus limitaciones.

Relación con PAD-Bienestar: El sistema necesita registrar qué equipos están disponibles, quién tiene una sesión activa, quién espera turno y qué eventos deben conservarse en una bitácora.

Necesidad del sistema	Estructura en C++	Razón principal
Controlar equipos	Arreglo	Acceso directo por índice.
Representar una sesión	struct	Agrupar datos relacionados.
Guardar varias sesiones	Arreglo de struct	Maneja varios registros.
Registrar eventos del día	Lista estática	Usa un arreglo y un contador.
Consultar acciones recientes	Pila	Trabaja con lo último registrado.
Administrar turnos	Cola	Respetar el orden de llegada.
Conservar bitácoras	Archivos	Guarda datos entre ejecuciones.

2. Arreglos en C++

Teoría orientada a C++

Un arreglo es una estructura lineal estática que almacena varios elementos del mismo tipo en posiciones consecutivas de memoria. En C++, los índices comienzan en 0, por lo que un arreglo de 10 elementos tiene posiciones de 0 a 9.

Su principal ventaja es el acceso directo: si se conoce la posición del dato, se puede consultar rápidamente usando el índice. Esto lo vuelve útil cuando el tamaño de la información se conoce desde el inicio.

Su limitación principal es el tamaño fijo. Una vez declarado, no puede crecer automáticamente durante la ejecución del programa.

Aplicación PAD-Bienestar: En PAD-Bienestar, un arreglo puede representar el estado de los equipos disponibles. Por ejemplo: 0 = disponible, 1 = ocupado y 2 = mantenimiento.

Ejemplo base en C++

```
int equipos[10] = {0};

// 0 = disponible
// 1 = ocupado
// 2 = mantenimiento

equipos[0] = 1; // Equipo 1 ocupado
equipos[3] = 2; // Equipo 4 en mantenimiento
```

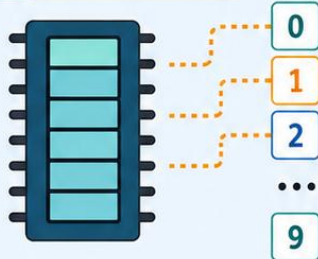
Imagen de apoyo visual



Arreglos en C++



1 ¿Qué es?



Un arreglo almacena varios datos del mismo tipo en posiciones consecutivas de memoria. Su tamaño es fijo y permite acceso rápido por índice.

2 Ejemplo en C++

```
int equipos[10] = {0};
// 0 = disponible
// 1 = ocupado
// 2 = mantenimiento
```

Representación del arreglo (equipos)

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
0	1	0	2	0	0	1	2	0	0
0 Disponible	1 Ocupado	2 Mantenimiento							

3 Caso PAD-Bienestar



Cada posición del arreglo representa el estado de un equipo del PAD-Bienestar.

0	Disponible
1	Ocupado
2	Mantenimiento



Ventaja:
acceso directo

Podemos acceder a cualquier elemento del arreglo usando su índice en tiempo constante $O(1)$.



Limitación:
tamaño fijo

El tamaño del arreglo se define al crearlo y no puede cambiar durante la ejecución del programa.

Imagen 1. Arreglos en C++ aplicados al estado de equipos del PAD-Bienestar.

3. Struct en C++

Teoría orientada a C++

Un struct permite crear un tipo de dato definido por el usuario. Su función es agrupar varios campos relacionados, incluso si son de distintos tipos.

En C++, un struct ayuda a modelar entidades del mundo real. En lugar de tener variables sueltas para id, nombre, equipo y estado, se puede representar todo como una sola unidad lógica.

Esto mejora la legibilidad del programa porque cada conjunto de datos tiene un significado claro dentro del problema.

Aplicación PAD-Bienestar: En PAD-Bienestar, una sesión activa puede representarse con un struct que almacene el identificador del usuario, el nombre, el equipo asignado y si la sesión está activa.

Ejemplo base en C++

```
struct Sesion {  
    int idUsuario;  
    string nombreUsuario;  
    int equipoAsignado;  
    bool activa;  
};  
  
Sesion sesion1;  
sesion1.idUsuario = 101;  
sesion1.nombreUsuario = "Ana Lopez";  
sesion1.equipoAsignado = 3;  
sesion1.activa = true;
```

Imagen de apoyo visual



Struct en C++



1 ¿Qué es?

REGISTRO DE SESIÓN		
	ID	101
	Nombre	Ana López
	Equipo	3
	Estado	Activa

Un struct agrupa datos relacionados de distintos tipos bajo un mismo nombre.

Permite modelar entidades del mundo real de forma organizada.

2 Ejemplo en C++

```
struct Sesion {
    int idUsuario;
    string nombreUsuario;
    int equipoAsignado;
    bool activa;
};
```

Acceso a un campo

```
sesion1.equipoAsignado = 3;
```

idUsuario : identificador único del usuario
nombreUsuario : nombre del usuario
equipoAsignado : número del equipo asignado
activa : estado de la sesión (true/false)

3 Caso PAD-Bienestar



Cada sesión activa puede representarse como un struct con los datos del usuario, el equipo asignado y el estado de la sesión.

Ejemplo de sesión actual

```
Sesion sesion1;
sesion1.idUsuario = 101;
sesion1.nombreUsuario = "Ana López";
sesion1.equipoAsignado = 3;
sesion1.activa = true;
```



Ventaja:
organiza información relacionada.



Limitación:
una sola estructura representa un registro; para muchas sesiones se necesita combinarla con arreglos u otras estructuras.

Imagen 2. Struct en C++ para representar una sesión activa del PAD-Bienestar.

4. Arreglos de estructuras en C++

Teoría orientada a C++

Un arreglo de estructuras combina dos ideas: el almacenamiento secuencial de un arreglo y la organización interna de un struct.

Cada posición del arreglo almacena un registro completo. Esto permite manejar colecciones de entidades, como varias sesiones, varios usuarios o varios eventos.

Es una solución práctica para una primera versión del sistema, siempre que se conozca el número máximo de registros que se desea manejar.

Aplicación PAD-Bienestar: En PAD-Bienestar, un arreglo de estructuras permite guardar varias sesiones activas en un solo contenedor, donde cada índice representa una sesión distinta.

Ejemplo base en C++

```
struct Sesion {
    int idUsuario;
    string nombreUsuario;
    int equipoAsignado;
    bool activa;
};

Sesion sesiones[3];
sesiones[0] = {101, "Ana Lopez", 1, true};
sesiones[1] = {102, "Carlos Perez", 2, true};
sesiones[2] = {103, "Maria Ruiz", 3, false};
```


Imagen de apoyo visual



Arreglos de estructuras en C++



1 ¿Qué es?

Un arreglo de estructuras almacena varios registros del mismo tipo. Cada posición guarda una estructura completa.

[0]	[1]	[2]
Sesión 0 # idUsuario: 101 nombre: Ana Lopez equipo: 1 activa: true	Sesión 1 # idUsuario: 102 nombre: Carlos Perez equipo: 2 activa: true	Sesión 2 # idUsuario: 103 nombre: Maria Ruiz equipo: 3 activa: false

2 Ejemplo en C++

```
struct Sesion {
    int idUsuario;
    string nombreUsuario;
    int equipoAsignado;
    bool activa;
};

Sesion sesiones[3];
sesiones[0] = {101, "Ana Lopez", 1, true};
sesiones[1] = {102, "Carlos Perez", 2, true};
sesiones[2] = {103, "Maria Ruiz", 3, false};
```

3 Caso PAD-Bienestar

SESIONES ACTIVAS		
	ID: 101 Nombre: Ana Lopez Equipo: 1	✓ Activa
	ID: 102 Nombre: Carlos Perez Equipo: 2	✓ Activa
	ID: 103 Nombre: Maria Ruiz Equipo: 3	✗ Inactiva

El sistema puede guardar varias sesiones activas en un solo arreglo. Cada índice representa una sesión distinta.



✓ Activa
✗ Inactiva



Ventaja:
permite manejar
varios registros



Limitación:
tamaño fijo

Imagen 3. Arreglo de estructuras para administrar varias sesiones del PAD-Bienestar.

5. Lista estática en C++

Teoría orientada a C++

Una lista estática puede implementarse con un arreglo y una variable contador que indique cuántos elementos están ocupados.

Aunque internamente usa un arreglo, se maneja como una lista porque se controla qué posiciones están usadas y cuáles siguen disponibles.

Permite insertar, consultar, recorrer y eliminar elementos, pero las inserciones o eliminaciones en posiciones intermedias pueden requerir desplazamientos.

Aplicación PAD-Bienestar: En PAD-Bienestar, una lista estática puede servir para almacenar las bitácoras del día: inicios de sesión, cierres, reportes de mantenimiento o cambios de estado.

Ejemplo base en C++

```
struct Bitacora {  
    int idEvento;  
    string descripcion;  
    string fecha;  
};  
  
const int MAX_EVENTOS = 5;  
Bitacora eventos[MAX_EVENTOS];  
int totalEventos = 0;  
  
eventos[totalEventos] = {1, "Usuario inicio sesion", "25/05/2026"};  
totalEventos++;
```

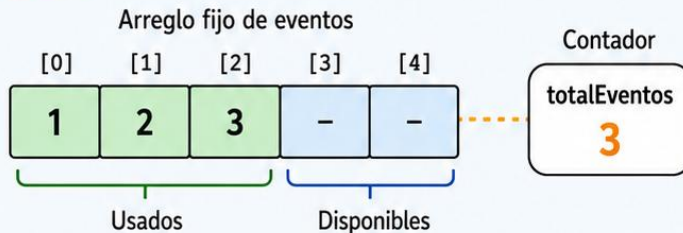
Imagen de apoyo visual



Lista estática en C++



1 ¿Qué es?



Una lista estática se implementa con un arreglo y una variable que indica cuántos elementos hay almacenados.

Permite insertar, consultar, recorrer y eliminar dentro de una capacidad fija.

2 Ejemplo en C++

```
struct Bitacora {
    int idEvento;
    string descripcion;
    string fecha;
};

const int MAX_EVENTOS = 5;
Bitacora eventos[MAX_EVENTOS];
int totalEventos = 0;

eventos[totalEventos] = {1, "Usuario inicio sesion", "25/05/2026"};
totalEventos++;
```

3 Caso PAD-Bienestar

La lista estática puede almacenar bitácoras del día, como inicios de sesión, cierres o reportes de mantenimiento.

ID	Descripción	Fecha
1	Usuario inicio sesión	25/05/2026 08:15
2	Generación de reporte	25/05/2026 10:30
3	Cierre de sesión	25/05/2026 17:45
-	-	-

Usados (hasta totalEventos)

Disponibles (no utilizados)

totalEventos Cantidad actual de eventos



Ventaja:
implementación sencilla.



Limitación:
inserciones y eliminaciones pueden requerir desplazamientos.

Imagen 4. Lista estática para registrar eventos diarios del PAD-Bienestar.

6. Pila en C++

Teoría orientada a C++

Una pila es una estructura lineal que sigue el principio LIFO: Last In, First Out. Esto significa que el último elemento en entrar es el primero en salir.

En una pila, las operaciones principales se realizan sobre un extremo llamado tope. Las operaciones típicas son push para insertar, pop para retirar y peek o top para consultar el elemento superior.

La pila es útil cuando el programa necesita trabajar con la información más reciente.

Aplicación PAD-Bienestar: En PAD-Bienestar, una pila puede usarse para consultar la última acción registrada, como el último inicio de sesión, el último cambio de estado o el último reporte generado.

Ejemplo base en C++

```
const int MAX = 5;
string pila[MAX];
int tope = -1;

// Push
tope++;
pila[tope] = "Inicio de sesion usuario 101";

tope++;
pila[tope] = "Asignacion de equipo 2";

// Peek
cout << pila[tope];
```

Imagen de apoyo visual

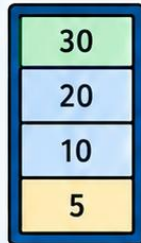


Pila en C++



1 ¿Qué es?

tope →



Push
(apilar)

Pop
(desapilar)

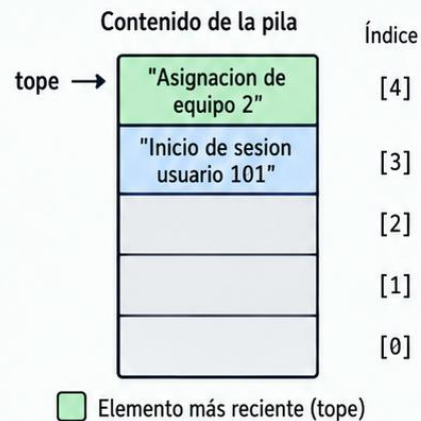
Una pila es una estructura lineal que sigue el principio **LIFO**: el último en entrar es el primero en salir.

2 Ejemplo en C++

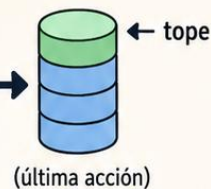
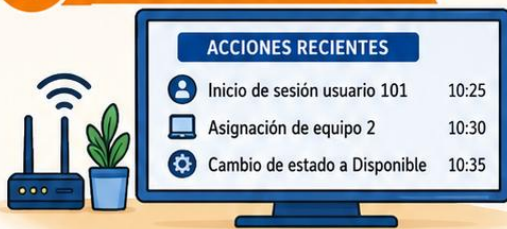
```
const int MAX = 5;
string pila[MAX];
int tope = -1;

// Push
tope++;
pila[tope] = "Inicio de sesion usuario 101";
tope++;
pila[tope] = "Asignacion de equipo 2";

// Peek
cout << pila[tope];
```



3 Caso PAD-Bienestar



La pila puede usarse para consultar la última acción registrada, como el último inicio de sesión o el último cambio de estado.



Ventaja:
acceso rápido al elemento más reciente



Limitación:
solo se trabaja con el tope

Imagen 5. Pila en C++ para consultar acciones recientes del sistema.

7. Cola en C++

Teoría orientada a C++

Una cola es una estructura lineal que sigue el principio FIFO: First In, First Out. Esto significa que el primer elemento en entrar es el primero en salir.

La inserción se realiza al final de la cola y la eliminación se realiza por el frente. Por eso es una estructura adecuada para administrar turnos.

Su principal ventaja es que refleja una lógica justa y ordenada de atención; su limitación es que el acceso se restringe al frente y al final.

Aplicación PAD-Bienestar: En PAD-Bienestar, la cola permite organizar los turnos de atención de las personas usuarias respetando el orden de llegada.

Ejemplo base en C++

```
const int MAX = 5;
string cola[MAX];
int frente = 0;
int final = -1;

final++;
cola[final] = "Ana";
final++;
cola[final] = "Carlos";
final++;
cola[final] = "Maria";

cout << cola[frente];
```


Imagen de apoyo visual



Cola en C++



1 ¿Qué es?

Una cola es una estructura lineal que sigue el principio FIFO: el primero en entrar es el primero en salir.



2 Ejemplo en C++

```
const int MAX = 5;
string cola[MAX];
int frente = 0;
int final = -1;

final++;
cola[final] = "Ana";
final++;
cola[final] = "Carlos";
final++;
cola[final] = "Maria";
cout << cola[frente];
```

Estado de la cola (MAX = 5)



☒ Elementos en la cola

☐ Posiciones vacías

3 Caso PAD-Bienestar



La cola organiza los turnos de atención del PAD-Bienestar respetando el orden de llegada.

☒ Elementos en la cola

☐ Espacios disponibles

Orden de atención (FIFO)



Ventaja:
atención justa
y ordenada.



Limitación: acceso restringido
a frente y final.

Imagen 6. Cola en C++ para administrar turnos de atención del PAD-Bienestar.

8. Archivos para persistencia en C++

Teoría orientada a C++

La persistencia permite conservar información después de que el programa termina su ejecución. Sin persistencia, los datos guardados en memoria se pierden al cerrar el programa.

En C++, el manejo de archivos se realiza mediante flujos. ofstream permite escribir información en un archivo; ifstream permite leer información desde un archivo; fstream puede utilizarse para operaciones de lectura y escritura.

Esta parte es esencial para sistemas que necesitan conservar bitácoras, reportes o registros históricos.

Aplicación PAD-Bienestar: En PAD-Bienestar, las bitácoras pueden guardarse en un archivo de texto para consultarlas después y analizar el uso del servicio.

Ejemplo base en C++

```
#include <fstream>

ofstream archivo("bitacora_pad.txt");
archivo << "Usuario 101 inicio sesion en equipo 1" << endl;
archivo << "Equipo 4 reportado en mantenimiento" << endl;
archivo.close();

ifstream lectura("bitacora_pad.txt");
string linea;
while (getline(lectura, linea)) {
    cout << linea << endl;
}
```

Imagen de apoyo visual



Archivos para persistencia en C++



1 ¿Qué es?



Programa



Archivo .txt



Lectura posterior

La persistencia permite guardar información para que no se pierda al cerrar el programa.

En C++ se trabaja con archivos usando **ifstream** y **ofstream**.

2 Ejemplo en C++

Escritura (guardar)

```
ofstream archivo("bitacora_pad.txt");
archivo << "Usuario 101 inicio sesion en equipo 1" << endl;
archivo << "Equipo 4 reportado en mantenimiento" << endl;
archivo.close();
```

Lectura (obtener)

```
ifstream lectura("bitacora_pad.txt");
string linea;
while (getline(lectura, linea)) {
    cout << linea << endl;
}
```

3 Caso PAD-Bienestar



```
bitacora_pad.txt
2025-05-12 09:15:23
Usuario 101 inicio sesion en equipo 1
2025-05-12 10:02:11
Equipo 4 reportado en mantenimiento
2025-05-12 11:30:45
Usuario 205 finaliza sesion
2025-05-12 12:05:10
Servicio de internet estable
```



Las bitácoras del PAD-Bienestar pueden guardarse en archivos para consultarlas después y analizar el uso del servicio.

- ✓ Registro de eventos
- ✓ Consulta posterior
- ✓ Análisis de uso



Ventaja:

conserva datos entre ejecuciones



Limitación:

requiere manejo correcto de apertura y cierre de archivos

Imagen 7. Archivos en C++ para conservar bitácoras del PAD-Bienestar.

9. Cierre didáctico

Conclusión clave: las estructuras lineales estáticas en C++ son una excelente primera aproximación para organizar información cuando se conoce un límite máximo de datos. Permiten construir soluciones claras para controlar equipos, sesiones, turnos y bitácoras. Sin embargo, como su tamaño es fijo, no son la mejor opción cuando el volumen de información crece de forma impredecible.

Puente hacia módulos posteriores: Este módulo prepara la base para comprender memoria dinámica y estructuras enlazadas, que permiten crear soluciones más flexibles para problemas reales.

Fuentes de apoyo

Universidad Nacional Rosario Castellanos. (2026). Estructura de Datos. Módulo 1: Fundamentos y estructuras lineales estáticas. Contenido Nuclear.

Deitel, P. J., & Deitel, H. M. (2014). C++ Cómo programar (9.^a ed.). Pearson.

Joyanes Aguilar, L. (2017). Fundamentos de programación: algoritmos, estructuras de datos y objetos (4.^a ed.). McGraw-Hill.